

Linguagem de Programação Orientada a Objetos

NOTAS DE AULA

Paulo A. Pagliosa

Capítulo 4 Introdução a Java

Java é um sistema de programação orientado a objetos (POO) caracterizado por uma linguagem e seu compilador, um conjunto de bibliotecas de classes, ou API (*application programming interface*), e uma máquina virtual.

A API Java é uma coleção de *pacotes*, cada pacote uma coleção de classes compiladas. A API que acompanha o JDK (*Java development kit*) é bastante extensa, com pacotes de classes para programação de entrada/saída (incluindo arquivos), estruturas de dados, funções matemáticas, comunicação via rede, interfaces gráficas, acesso a banco de dados, persistência de objetos, programação distribuída, gráficos 2D e 3D, reflexão computacional, web, entre muitas outras coisas. A máquina virtual Java (JVM, *Java virtual machine*) é um programa responsável pela carga de um arquivo Java compilado e pela execução do código objeto deste arquivo.

Características de Java

- Java é dito independente de plataforma e arquitetura porque um programa Java é executado por outro programa que é a JVM. Então, a independência só ocorre se houver uma JVM disponível para determinada plataforma e arquitetura.
- Java é compilada e interpretada. O compilador Java toma como entrada um programa fonte codificado em um arquivo texto com a extensão `.java` e gera como saída um arquivo com o mesmo nome, mas com a extensão `.class`. Este arquivo contém o código objeto gerado pelo compilador, o qual consiste de uma sequência de bytes, cada byte representando uma operação que pode ou não ter outros bytes da sequência como argumentos. Esta sequência de bytes é chamada *bytecode*. Cada método em uma classe Java terá o seu *bytecode* em um arquivo `.class`. A JVM carrega o arquivo `.class` e interpreta o *bytecode*.

4.1 Resumo da Sintaxe da Linguagem Java

Um arquivo `.java` é denominado *unidade de compilação*. Em uma unidade de compilação, podem ser declarados, na ordem:

1. Importação de classes (opcional);
2. Tipos, os quais podem ser *classes* ou *interfaces*.

A sintaxe é:

```
UnidadeDeCompilação:
    DeclaraçãoDeImportação*
    DeclaraçãoDeTipo*
```

Uma declaração de importação define quais classes de quais pacotes serão utilizadas ao longo da unidade de compilação. Há classes da API (tais como `java.lang.Object`, `java.lang.String` e `java.lang.Exception`) que são automaticamente importadas pelo compilador, não sendo necessário explicitar a declaração de importação. A sintaxe é

```
DeclaraçãoDeImportação:
    IMPORT Nome ";"
| IMPORT Nome "." "*" ";"
```

onde `IMPORT` é a palavra reservada **import** e `Nome` é o *nome qualificado* da classe ou pacote. Um nome qualificado `Q.s` é formado pela concatenação do nome qualificado `Q` (chamado *qualificador*) seguido de ponto e do *nome simples* dado pelo identificador `s`:

```
Nome:
    IDENTIFICADOR ("." IDENTIFICADOR)*
```

Uma declaração de tipo pode ser uma declaração de classe ou de interface.

```
DeclaraçãoDeTipo:
    DeclaraçãoDeClasse | DeclaraçãoDeInterface
```

A declaração de uma classe com nome simples `IDENTIFICADOR` e superclasse `Nome` em Java obedece a seguinte sintaxe:

```
DeclaraçãoDeTipo:
    Modificador* CLASS IDENTIFICADOR (EXTENDS Nome)?
    "{"
        Membro*
    "}"
```

onde `CLASS` e `EXTENDS` são as palavras reservadas **class** e **extends**, respectivamente. Os modificadores podem ser:

```
Modificador:
    PUBLIC
| PROTECTED
| PRIVATE
| STATIC
| ABSTRACT
| FINAL
| SYNCHRONIZED
```

correspondentes às palavras reservadas: **public**, **protected**, **private**, **static**, **abstract**, **final** e **synchronized**. Em uma declaração de classe são válidos os modificadores:

- `PUBLIC`, indicando que a visibilidade da classe é pública, ou seja, pode ser acessada de qualquer outra unidade de compilação (se omitido a visibilidade da classe é restrita ao pacote no qual a unidade de compilação foi declarada).

- `ABSTRACT`, indicando que a classe é abstrata (independentemente desta possuir ou não métodos abstratos).
- `FINAL`, indicando que a classe não pode ser derivada, ou seja, não pode ser superclasse de outra.

Em Java, apenas herança simples é admitida. Se a superclasse não for especificada, a classe sendo declarada deriva implicitamente da classe `java.lang.Object`.

Um membro de uma classe pode ser uma declaração de: atributo, método, construtor ou de um tipo interno:

```
Membro:
  DeclaraçãoDeAtributo
| DeclaraçãoDeMétodo
| DeclaraçãoDeConstrutor
| DeclaraçãoDeTipo
```

(Em Java não há declaração de destrutor: todo objeto que não pode ser alcançado pela JVM é automaticamente destruído pelo coletor de lixo da máquina virtual.) A declaração de um atributo com nome simples `IDENTIFICADOR` obedece a seguinte sintaxe:

```
DeclaraçãoDeAtributo:
  Modificador* Declaração

Declaração:
  Tipo Declaradores ";"

Declaradores:
  Declarador ("," Declarador)*

Declarador:
  IDENTIFICADOR ("[" "]"")* Inicializador?
```

Os modificadores válidos na declaração de um atributo são:

- `PUBLIC`, `PROTECTED` ou `PRIVATE`, indicando a visibilidade do atributo (a ausência indica que o atributo é visível no pacote onde a classe é declarada).
- `FINAL`, indicando que o atributo é uma constante, ou seja, seu valor não pode ser modificado (neste caso, `Inicializador` deve ser obrigatório).
- `STATIC`, indicando que o atributo é um atributo de classe.

O tipo de um atributo pode ser um tipo definido pelo programador (classe) ou um tipo primitivo definido pelas seguintes palavras reservadas da linguagem:

- **`boolean`**: `true` ou `false`;
- **`byte`**: número inteiro com sinal de 8 bits;
- **`char`**: caractere UNICODE de 16 bits;
- **`double`**: número real de 64 bits;
- **`float`**: número real de 32 bits;
- **`int`**: número inteiro com sinal de 32 bits;

- **long**: número inteiro com sinal de 64 bits;
- **short**: número inteiro com sinal de 16 bits.

Além de classes e tipos primitivos, outros tipos podem ser definidos através de vetores uni ou multidimensionais. Por exemplo, `int[]` representa o tipo vetor de inteiros e `x[][]` representa matriz de objetos da classe `x`. Em Java, uma instância de um tipo vetorado é um objeto daquele tipo. A sintaxe de tipo é:

```
Tipo:
    NomeDeTipo ("[" "]"*)

NomeDeTipo:
    NomeDeTipoPrimitivo
| Nome // nome de classe

NomeDeTipoPrimitivo:
    BOOLEAN
| BYTE
| CHAR
| DOUBLE
| FLOAT
| INT
| LONG
| SHORT
```

Os nomes de tipos primitivos correspondem às palavras reservadas: **boolean**, **byte**, **char**, **double**, **float**, **int**, **long** e **short**.

O inicializador define o valor inicial do atributo (obrigatório se o atributo é final). Pode ser um inicializador simples ou, se o atributo for de um tipo vetorado, um inicializador de vetor. A sintaxe é:

```
Inicializador:
    "=" (InicializadorSimples | InicializadorDeVetor)

InicializadorSimples:
    Expressão

InicializadorDeVetor:
    "{" ListaDeElementosDeVetor? "}"

ListaDeElementosDeVetor:
    ElementosDeVetor ("," ElementosDeVetor)*

ElementosDeVetor:
    Expressão
| InicializadorDeVetor
```

A sintaxe de declaração de um método com nome simples IDENTIFICADOR é:

```
DeclaraçãoDeMétodo:
    Modificador* TipoDeRetorno IDENTIFICADOR "(" Parâmetros? ")"
    EspecificadorDeExceções?
    (CorpoDeMétodo | ";")
```

```
TipoDeRetorno:
    Tipo | VOID
```

onde `VOID` é a palavra reservada `void` (se usada na especificação do tipo de retorno, indica que o método não possui valor de retorno). Os modificadores válidos na declaração de um atributo são:

- `PUBLIC`, `PROTECTED` ou `PRIVATE`, indicando a visibilidade do método (a ausência indica que o método é visível no pacote onde a classe é declarada).
- `FINAL`, indicando que o método não pode ser sobrescrito em classes derivadas. (Em Java, todos os métodos não estáticos, exceto os construtores, são virtuais). Se a classe foi declarada como `final`, então todos os seus métodos são finais.
- `STATIC`, indicando que o método é um método de classe.
- `ABSTRACT`, indicando que o método é abstrato. Neste caso, a classe também tem que ser declarada com `abstract`. Um método abstrato não pode ser `final` ou `estático`.

A declaração dos parâmetros do método obedece a seguinte sintaxe:

```
Parâmetros:
    Parâmetro ("," Parâmetro)*
```

```
Parâmetro:
    FINAL? Tipo IDENTIFICADOR
```

(`FINAL` indica que o valor do parâmetro é constante dentro do corpo do método) Se o método pode lançar exceções (veja Seção 4.2), a lista com os nomes das classes de exceções que podem ser lançadas no método devem ser informadas no especificador de exceções:

```
EspecificadorDeExceções:
    THROWS Nome ("," Nome)*
```

Se o método foi declarado como abstrato, então não possui corpo e sua declaração termina com ponto e vírgula. Caso contrário, a declaração termina com o bloco que define o corpo do método:

```
CorpoDeMétodo:
    Bloco
```

```
Bloco:
    "{"
    Sentença*
    "}"
```

A sintaxe de sentenças e expressões em Java são muito similares a do C++. A sintaxe comum de sentenças é:

```
Sentença:
    Declaração
| Bloco
| Expressão? ";"
| SentençaIf
```

```
| SentençaWhile  
| SentençaDo  
| SentençaFor  
| SentençaSwitch  
| SentençaCase  
| SentençaBreak  
| SentençaContinue  
| SentençaReturn  
| SentençaThrow  
| SentençaTry  
| SentençaCatch
```

(Há outras sentenças específicas de Java que não serão apresentadas aqui). Apenas para exemplificar, a sintaxe de sentenças tipo IF-ELSE é:

```
SentençaIf:  
IF "(" Expressão ")" Sentença (ELSE Sentença)?
```

onde IF e ELSE são as palavras reservadas **if** e **else**, respectivamente.

A declaração de um construtor da classe IDENTIFICADOR obedece a seguinte sintaxe:

```
DeclaraçãoDeConstrutor:  
Modificador* IDENTIFICADOR "(" Parâmetros? ")"  
EspecificadorDeExceções?  
CorpoDeConstrutor
```

Os únicos modificadores válidos na declaração de um construtor são: **PUBLIC**, **PROTECTED** ou **PRIVATE**, indicando a visibilidade do método (a ausência indica que o construtor é visível no pacote onde a classe é declarada). O corpo do construtor é um bloco, como o corpo de um método, mas com a diferença que a primeira sentença pode ser a invocação de um dos construtores da superclasse (palavra reservada **super**) ou a invocação de outro construtor da classe (palavra reservada **this**). A sintaxe é:

```
CorpoDeConstrutor:  
"{ "  
    ((SUPER | THIS) "(" ListaDeExpressões ")" ";" )?  
    Sentença*  
"} "
```

```
ListaDeExpressões:  
Expressão ("," Expressão)*
```

onde **SUPER** e **THIS** são as palavras reservadas **super** e **this**, respectivamente, e a lista de expressões são os argumentos do construtor sendo invocado.

Uma unidade de compilação Java pode ter várias declarações de tipo, mas deve declarar, na prática, uma classe cujo nome é o mesmo nome da unidade de compilação. Esta classe principal deve ser a única classe pública da unidade de compilação.

O ponto de entrada de execução de uma aplicação Java é o método declarado na classe principal da unidade de compilação cuja assinatura é:

```
static public void main (String[] args)
```

O parâmetro `args` são os argumentos passados para o método na linha de comando.

Exercício 4.1

Implemente uma lista de objetos em Java com métodos para adicionar, procurar e remover elementos, bem como obter um iterador para a lista.

Exercício 4.2

Estude as classes `java.lang.Object`, `java.lang.String` e `java.lang.Exception`.

4.2 Tratamento de Exceções

Em Java, uma exceção definida pelo programador é um objeto da classe `Exception` ou dela derivada. Como em C++, sentenças de um método que podem resultar no lançamento de uma ou mais exceções ou (1) devem ser enclausuradas no bloco de uma sentença **try** no corpo do método, ou (2) o método deve especificar em seu cabeçalho quais exceções este pode lançar (veja a sintaxe de declaração de métodos na Seção 4.1).

Se uma exceção é lançada dentro de um bloco **try**, o fluxo de controle é transferido para a primeira sentença do primeiro bloco **catch** cujo parâmetro casar com o tipo da exceção lançada. Por exemplo:

```
try
{
    // tenta executar algumas sentenças aqui
    ...
}
catch (IOException e)
{
    // trata erro de entrada e saída
    ...
}
catch (ClassCastException e)
{
    // trata erro de conversão de tipo
    ...
}
```

(`IOException` e `ClassCastException` derivam ambas de `Exception`). Observe agora:

```
try
{
    // tenta executar algumas sentenças aqui
    ...
}
catch (Exception e)
{
    // trata QUALQUER exceção
    ...
}
```

```

catch (IOException e)
{
    // trata erro de entrada e saída
    ...
}
catch (ClassCastException e)
{
    // trata erro de conversão de tipo
    ...
}

```

Independentemente da exceção lançada no bloco **try**, o bloco a ser executado seria aquele do primeiro **catch**, pois é claro que qualquer exceção verificada é do tipo `Exception`. Se uma exceção lançada não casar em tipo ou parâmetro de nenhum **catch**, esta será tratada internamente pela JVM.

Java ainda permite o uso opcional de uma sentença **finally**, cujo bloco deve suceder o último **catch** (a presença do bloco **finally** torna opcional a presença de blocos **catch**). Se um bloco **finally** estiver presente, suas sentenças sempre serão executadas pela JVM, independentemente de uma exceção ter ou não sido lançada no **try** e tratada em algum **catch**.

4.2 Interfaces

Em Java, pode-se definir um novo tipo através de uma declaração de classe. Além da declaração de classe, um novo tipo pode ser definido através de uma declaração de interface. Uma declaração de interface é similar a uma declaração de classe contendo apenas atributos estáticos constantes e métodos abstratos, como no exemplo a seguir:

```

public interface I
{
    public void m1 ();
    public void m2 ();
}

```

A interface `I` acima declara métodos públicos `m1()` e `m2()` (não é necessário uso explícito do modificador **abstract**). Não é proibida a declaração de atributos em uma interface, mas se houver esta deve ser **final** e **static**; não há atributos de instância em uma interface. A sintaxe de declaração de interface é:

```

DeclaraçãoDeInterface:
Modificador* INTERFACE IDENTIFICADOR (EXTENDS Nome)?
"{"
    Membro*
"}"

```

onde `INTERFACE` é a palavra reservada **interface**, `Nome` é o nome qualificado de outra interface (note: uma interface pode entender outra interface, não uma classe) e `Membro` é uma declaração de método (abstrato) ou de atributo (**final static**).

Uma classe Java pode implementar uma ou mais interfaces. Implementar uma interface significa que todos os métodos declarados na interface devem ser sobrecarregados na classe. Note: uma classe pode entender somente outra classe (herança simples), mas pode implementar várias interfaces. Interfaces possibilitam a adição de múltiplas funcionalidades a uma classe Java. Um objeto de uma classe x que implementa uma interface I é efetivamente do tipo x e também do tipo I .

A sintaxe revista de declaração de tipo é:

```
DeclaraçãoDeTipo:
  Modificador* (CLASS | INTERFACE) IDENTIFICADOR
  (EXTENDS Nome)?
  (IMPLEMENTS ListaDeNomes)?
  "{"
  Membro*
  "}"
```

```
ListaDeNomes:
  Nome ("," Nome)*
```

onde `IMPLEMENTS` é a palavra reservada **implements**. O(s) nome(s) em `ListaDeNomes` deve(m) ser nome(s) de interface(s).

Exemplo 4.1

A classe `A` abaixo estende a classe `X` e implementa a interface `I`.

```
public class A extends X implements I
{
    public void m1 ()
    {
        ...
    }
    public void m2 ()
    {
        ...
    }
    ...
}
```

Pode-se escrever `x = new A()`, pois o objeto `x` é do tipo `x`, uma vez que a classe `A` deriva ou entende a classe `x`. Pode-se escrever também `i = new A()`, pois o objeto `i`, um novo `A`, é do tipo `I` uma vez que a classe `A` implementa a interface `I`.

Tem-se também a seguinte equivalência:

```
public abstract class X
{
    public abstract void m1 ();
    public abstract void m2 ();
}
```

é equivalente a:

```
public interface I
{
    public void m1 ();
    public void m2 ();
}

public abstract class A extends X implements I
{
}
```